

Technical Research Dossier: Edge AI Hub Integration (Jetson Orin Nano & Rubik Pi 3)

1. Hardware Node Profile and Identity

The N0.V4 (NeuroOmni / VagAgenti) architecture utilizes a bifurcated hub system to offload compute-intensive tasks from the primary edge device (Razr Ultra 2025). This configuration segregates heavy training and persistent data services from real-time tool execution and qualitative auditing. | **Feature** | Primary Hub: Jetson Orin Nano Super | Secondary Hub: Rubik Pi 3 || ----- | ----- | ----- || **SoC / GPU / NPU** | 1024 CUDA cores (67 TOPS) | Qualcomm Dragonwing QCS6490 / Hexagon 770 (12 TOPS) || **RAM** | 8 GB unified LPDDR5 | 12 GB LPDDR5 (Integrated) || **Storage** | Samsung 980 500GB Gen3 NVMe | Internal eMMC / SD support || **Network Transport** | Gigabit Ethernet (Tailscale Primary) | Gigabit Ethernet + Wi-Fi 6E + USB RNDIS || **OS Requirement** | JetPack 6.2.2 (via 5.1.3 firmware update path) | Ubuntu 24.04 Noble (Qualcomm kernel) || **Primary Role** | **Heavy Compute:** LoRA fine-tuning, PTL FastAPI server, Postgres + pgvector. | **Tool Execution:** Audit mirror, wiki host, Docker containers, secondary NPU experiments. |

2. LiteRT: Unified On-Device Inference Acceleration

Google's LiteRT (formerly TFLite) has graduated to a production-ready stack, serving as the core framework for accelerating machine learning models on N0.V4 hubs.

- **Acceleration Workflow:** LiteRT delivers a 1.4x GPU performance increase over legacy TFLite. For NPU acceleration, the framework employs a unified workflow: the **LiteRT NeuroPilot Stack** provides first-class target support for MediaTek Dimensity NPUs, while Qualcomm Hexagon NPUs are supported through the LiteRT NPU backend.
- **Extreme Quantization:** To support cross-platform GenAI for open models like **Gemma**, LiteRT introduces support for extreme bit-precision reduction:
- **INT2 and INT4 Support:** Now available for `tfl.cast`, `tfl.slice`, and `tfl.fully_connected` operators.
- **Enhanced Integer Math:** The SQRT and comparison operators natively support int8 and int16x8 precision.
- **Memory Impact:** These operators enable complex model deployment on memory-constrained edge nodes by drastically reducing weights and activation sizes.

3. Polar Framework: Token-Faithful Rollout and Training

NVIDIA's Polar framework is integrated into the Jetson node to facilitate Reinforcement Learning (RL) and trajectory reconstruction without modifying existing agent harnesses.

1. **Gateway Proxy Architecture:** Polar places a proxy at the model API boundary to perform a four-step capture process:
2. **Detection:** Identifies the provider API, distinguishing between Anthropic Messages, OpenAI Chat Completions, OpenAI Responses, and Google generateContent calls.
3. **Normalization:** Transforms requests/tools into the OpenAI Chat Completions shape for local inference.

4. **Token-level Capture:** Records messages, token IDs, log probabilities, and finish reasons.
5. **Provider Shape Return:** Re-transforms the response back into the specific schema expected by the agent harness.
6. **Trajectory Building Strategies:** | Strategy | Characteristics | Performance Impact | | :--- | :--- | :--- || **per_request** | Treats every model call as an independent trace. | Lossless but fragmented; susceptible to reward hacking. || **prefix_merging** | Reconstructs ordered chains by verifying token-prefix relations. | **5.39x Speedup** ; 87.7% GPU utilization (vs. 20.4%). |
7. **Harness Shortcuts:** Polar includes built-in shortcuts for rapid training integration with **codex**, **claude_code**, **qwen_code**, **opencode**, **pi**, and **gemini_cli** .

4. Framework-Agnostic Agent Logic with GitAgent

GitAgent provides a universal, framework-agnostic format to decouple agent definitions from specific orchestration environments like **LangChain**, **AutoGen**, **CrewAI**, **OpenAI Assistants**, and **Claude Code** .

- **Core Component Structure:**
- **agent.yaml:** Central manifest for metadata, model providers, and environment dependencies.
- **SOUL.md:** Declarative Markdown file defining identity, personality, and tone.
- **DUTIES.md:** Outlines responsibilities and the **Segregation of Duties (SOD)** framework.
- **memory/:** Stores long-term state in human-readable files (context.md, dailylog.md).
- **Git-Native Supervision:** Any update to an agent's memory or skill acquisition triggers a Git branch and a Pull Request (PR). This enables human-in-the-loop validation of behavioral shifts using standard CI/CD "diff" workflows.
- **Compliance and SOD:** The SOD framework enforces a conflict matrix (e.g., Maker, Checker, Executor roles) for regulated enterprise environments (FINRA/SEC). Configuration compliance is enforced via the gitagent validate command.

5. Multi-Agent Orchestration via SmolAgents

The SmolAgents framework is utilized for building multi-agent systems with dynamic Python execution.

- **Agent Paradigms:** The **CodeAgent** writes and executes Python logic for multi-step problems, while the **ToolCallingAgent** utilizes structured, ReAct-style reasoning loops.
- **Runtime Tool Injection:** Capabilities are extended dynamically through the agent.tools dictionary, allowing tool addition without rebuilding the agent.
- **Multi-Agent Hierarchy:** A **Manager Agent** coordinates specialized sub-agents, including:
 - **Math and Utilities Agent:** Discrete mathematical reasoning.
 - **Web Search Agent:** Real-time information retrieval.
 - **Stateful Memory Tool:** A class-based implementation for persistent, cross-step interaction data.

6. Cutting-Edge Model Integration and Placement

Models are deployed based on computational load and the specific hardware strengths of the hubs. **Model Deployment Vault:**

- **LFM2.5-Audio-1.5B (Q4: 1.2GB):** Unified speech evaluation and processing.
 - **MiniCPM-o 2.6 (8B) (Q4: 4.5GB):** Multimodal text/vision tasks when docked.
 - **Memory OS (6-Layer Stack):** Local persistent memory layer via Hermes Agent.
 - **Qwen3-4B (Q4: 2.5GB):** High-speed inference and tool-centric reasoning.
 - **Llama 3.1 8B (Q4: 4.5GB):** General-purpose complex reasoning.
- Placement Logic:**
- **Jetson (Primary Hub):** Heavy compute tasks including LoRA fine-tuning, the **Postgres + pgvector** database, and the **Prompt Translation Layer (PTL) FastAPI server**.
 - **Rubik Pi (Secondary Hub):** Light tool execution, audit mirroring, and local wiki hosting.

7. Transport and Connectivity Resilience

The network architecture prioritizes high-bandwidth wireless paths while maintaining a dedicated wired lifeline for reliability.

1. **Dual-Transport System:**
 2. **Primary:** Tailscale over Wi-Fi 6E/Ethernet. Wi-Fi 6E (1-2 Gbps) is the preferred data path for the Razr.
 3. **Secondary:** USB P2P Link/RNDIS Tethering (480 Mbps). This serves as a **guaranteed delivery** path for model/adaptor transfers and an **emergency SSH lifeline** for recovery.
- Resilience Matrix:** | Condition | System Status | | :--- | :--- | | **Home (Wi-Fi + Wired)** | Full stack active. All nodes connected via Tailscale mesh. | | **USB P2P Active** | Full stack active. Razr↔Pi direct link; routes to Jetson through Pi. | | **Edge Only (No Hub)** | Limited to local Razr models. No heavy compute. **Local JSONL logging active.** |

8. The Apprentice Learning & Audit Loop

The Jetson hub automates a sharpening cadence to optimize the system's dispatching accuracy over time.

- **Automated Auditing:**
- **Docker CLI #1 (Execution Auditor):** Specifically monitors the **Prompt Translation Layer (PTL)** server for latency, token counts, and specification drift.
- **Docker CLI #2 (Quality Auditor):** Analyzes qualitative **interaction logs** for issues such as sycophancy or contradictions, generating the JSONL reports required for fine-tuning.
- **Sharpening Cadence:** Performance-triggered cycles (Weeks 1-4: 5 days; Weeks 5-8: 10 days; Weeks 9+: 20 days). System deployment occurs at **8 weeks**.
- **Failure Taxonomy:** The auditors monitor for the following failure types:
- **Context Degradation:** Loss of session info across turns.
- **Specification Drift:** Implementation diverging from architecture.
- **Sycophantic Confirmation:** Unearned model agreement.
- **Silent Failure:** Success reported despite incorrect output.
- **Tool Selection Error:** Selection of inappropriate tools for the task.
- **Cascading Failure:** Propagation of errors through dependency chains.

